INSI 2026 — MASTER 1 INFORMATIQUE

Conception d'un Système ERP Orienté Services (SOA)

et Décisionnel (BI, Data Mining, Reporting et IA)

Application : Plateforme E-Commerce Madagascar

Équipe Projet

RAHOLDINA FIARA ANJARA MIHAVANA

ANDRIATSIFERANA NO KANTO LORIDA

RAKOTONIRINA MENDRIKA ITOKIANA

Année Universitaire 2025–2026

Master 1 — 4e année Informatique — I2AD

Table des Matières

1. Contexte et Problématique 3
2. Analyse des Besoins 5
3. Architecture Globale 8
4. ERP et Règles de Gestion 11
5. Architecture Orientée Services (SOA) 16
6. Business Intelligence, Data Mining et Reporting 19
7. Reporting Assisté par l'Intelligence Artificielle 25
8. Analyse Critique et Limites 28
9. Conclusion et Perspectives 31

1. Contexte et Problématique

1.1 Contexte Général

Le commerce électronique connaît une croissance exponentielle à Madagascar, porté par la pénétration rapide des téléphones mobiles et l'essor des systèmes de paiement mobile (Mvola, Orange Money, Airtel Money). Les PME malgaches font face à des défis considérables : gestion manuelle des stocks, traitement informel des commandes, absence de pilotage analytique et difficulté à fidéliser une clientèle de plus en plus mobile-first.

Dans ce contexte, notre projet vise à concevoir et implémenter une plateforme ERP complète, conçue spécifiquement pour l'écosystème malgache, combinée à une couche décisionnelle avancée s'appuyant sur la Business Intelligence, le Data Mining et l'Intelligence Artificielle. Le système est structuré selon une architecture orientée services (SOA), garantissant l'interopérabilité, la scalabilité et l'évolutivité.

1.2 Problématique

Les opérateurs du commerce en ligne malgache se trouvent confrontés à plusieurs problèmes structurels :

- Gestion des commandes : Processus manuels, risques d'erreurs, absence de traçabilité
- Gestion des stocks : Ruptures fréquentes, sur-stockage coûteux, absence d'alertes automatiques
- Gestion de la relation client : Absence de segmentation, politique commerciale uniforme sans personnalisation
- Gestion financière : Limites de crédit non contrôlées, risques d'impayés
- Pilotage décisionnel : Absence de tableaux de bord, décisions basées sur l'intuition plutôt que sur les données
- Analyse prédictive : Aucun outil de prévision des ventes ou de détection des comportements clients

Notre réponse à ces problèmes est la construction d'un système intégré couvrant l'ensemble de la chaîne opérationnelle et analytique, du traitement des transactions à la génération automatique de rapports stratégiques assistés par IA.

1.3 Périmètre du Projet

La plateforme Madagascar E-Commerce constitue notre contexte métier. Elle s'adresse à des PME de distribution commerciale opérant sur le marché malgache, vendant des produits artisanaux, alimentaires et agricoles. Les acteurs du système sont :

Acteur	Rôle dans le système
Administrateur	Gestion globale, configuration, supervision du système
Vendeur (Vendor)	Saisie des commandes, gestion du catalogue produits
Service Client	Suivi des commandes, gestion des réclamations
Client (Premium/Standard)	Passe des commandes, bénéficie de remises selon son statut
Décideur / Manager	Consultation des tableaux de bord et rapports IA

2. Analyse des Besoins

2.1 Besoins Fonctionnels

2.1.1 Module ERP Transactionnel

Le module ERP doit assurer la gestion complète du cycle de vie d'une commande, depuis sa création jusqu'à sa livraison. Les besoins identifiés incluent :

- Création et validation multi-étapes des commandes (PENDING → VALIDATED → DELIVERED)
- Vérification automatique de la limite de crédit client avant tout traitement
- Gestion des stocks en temps réel avec déduction automatique à la validation
- Système d'alertes sur stock de sécurité
- Gestion des utilisateurs avec contrôle d'accès basé sur les rôles (RBAC)
- Audit log complet pour traçabilité et conformité
- Support des paiements mobiles malgaches (Mvola, Orange Money, Airtel Money)

2.1.2 Module ETL et Data Warehouse

- Extraction automatisée des données opérationnelles depuis l'ERP
- Transformation et modélisation en schéma en étoile
- Chargement programmé vers le Data Warehouse
- Séparation stricte entre base opérationnelle (erp_db) et base analytique (bi_warehouse)

2.1.3 Module BI et Data Mining

- Segmentation client par l'analyse RFM (Récence, Fréquence, Montant)
- Prévision des ventes sur horizon de 3 mois
- Tableaux de bord interactifs et visualisations dynamiques
- Accès self-service via Métabase pour les non-techniciens

2.1.4 Module Reporting IA

- Génération automatique de rapports narratifs basés sur les KPIs
- Interprétation linguistique des indicateurs clés de performance
- Recommandations stratégiques formulées en langue naturelle

2.2 Besoins Non-Fonctionnels

Catégorie	Exigence
Disponibilité	Architecture Docker garantissant le redémarrage automatique des services
Sécurité	Isolation des bases de données, gestion des secrets via variables d'environnement
Scalabilité	Architecture microservices permettant la mise à l'échelle indépendante
Maintenabilité	Code modulaire, documentation Swagger auto-générée, services découplés
Performance	ETL optimisé par batch Pandas, API FastAPI asynchrone
Interopérabilité	API REST standardisée, format JSON pour tous les échanges

2.3 Modélisation des Cas d'Utilisation Principaux

Les cas d'utilisation principaux du système peuvent être regroupés en trois domaines :

Domaine opérationnel : La création d'une commande déclenche une chaîne de vérifications (crédit client, disponibilité produit), puis un workflow de validation qui entraîne la mise à jour du stock et la journalisation de chaque action dans le système d'audit.

Domaine analytique : L'ingénieur BI déclenche l'ETL qui alimente le Data Warehouse. Les analystes consultent ensuite les résultats RFM et les prévisions ARIMA via le dashboard Streamlit ou Métabase.

Domaine décisionnel : Le manager consulte les rapports générés par l'IA qui interprète les KPIs, formule des recommandations et produit une narration automatisée (data storytelling).

3. Architecture Globale

3.1 Vue d'Ensemble Architecturale

Le système est structuré en six services indépendants orchestrés par Docker Compose. Cette architecture garantit l'isolement des responsabilités, la facilité de déploiement et la résilience opérationnelle. Chaque service communique via des interfaces bien définies, conformément aux principes de l'architecture orientée services.

Service	Port	Technologie	Responsabilité
PostgreSQL (db)	5432	PostgreSQL 15	Stockage centralisé ERP + BI
ERP Service	8000	FastAPI + SQLAlchemy	Gestion opérationnelle
ETL Service	8002	FastAPI + Pandas	Pipeline de données
Analytics Service	8001	FastAPI + Scikit-learn	Data Mining + KPIs
Dashboard IA	8501	Streamlit + Groq	Interface décisionnelle
Metabase	3000	Metabase OSS	BI self-service

3.2 Architecture en Couches

Le système suit une architecture en quatre couches distinctes, chacune ayant des responsabilités spécifiques et communiquant uniquement avec les couches adjacentes :

Couche 1 — Présentation

Cette couche regroupe les interfaces utilisateur : le dashboard Streamlit pour les analystes et managers, Metabase pour l'exploration BI en libre-service, et la documentation Swagger auto-générée par FastAPI pour les développeurs.

Couche 2 — Services Métier (SOA)

Les trois microservices FastAPI (ERP, ETL, Analytics) constituent le cœur fonctionnel du système. Ils exposent des API REST consommées par la couche de présentation. Chaque service est autonome, avec sa propre logique métier et ses propres modèles de données.

Couche 3 — Accès aux Données

SQLAlchemy ORM assure l'abstraction de l'accès aux données pour l'ERP. Pandas gère les opérations bulk de l'ETL. Les deux bases de données (erp_db et bi_warehouse) sont physiquement séparées sur la même instance PostgreSQL.

Couche 4 — Persistance

PostgreSQL 15 constitue le système de gestion de base de données. Trois schémas logiques coexistent : erp_db pour les données opérationnelles, bi_warehouse pour l'entrepôt de données, et metabase_app_db pour la configuration interne de Métabase.

3.3 Flux de Données

Le flux de données suit un chemin unidirectionnel du système source vers les systèmes d'analyse, garantissant la séparation entre données opérationnelles et analytiques :

ERP (erp_db) → [ETL : Extraction + Transformation] → Data Warehouse (bi_warehouse) → [Analytics Service] → [Dashboard IA / Metabase] → Décideurs

L'ETL est programmé pour s'exécuter toutes les minutes via le scheduler Python, garantissant une fraîcheur des données quasi-temps-réel pour les tableaux de bord. Un endpoint REST /trigger permet également de déclencher manuellement le pipeline.

3.4 Sécurité et Configuration

La sécurité est gérée à plusieurs niveaux. Les credentials de base de données et le token Grok sont externalisés dans un fichier .env non versionné. Docker isole les services dans un réseau privé (bridge), et seuls les ports nécessaires sont exposés vers l'extérieur. SQLAlchemy protège contre les injections SQL grâce à l'utilisation de requêtes paramétrées.

4. ERP et Règles de Gestion

4.1 Modèles de Données

L'ERP repose sur six entités métier implémentées sous forme de tables SQLAlchemy ORM dans la base erp_db. Chaque entité correspond à un concept métier fondamental de la plateforme e-commerce malgache.

4.1.1 Entité User

L'entité User modélise les comptes administratifs du système. Chaque utilisateur dispose d'un identifiant unique, d'un nom d'utilisateur et d'un rôle (admin, vendor, customer_support). Cette entité est référencée dans les commandes et les journaux d'audit pour assurer la traçabilité des actions.

4.1.2 Entité Customer

L'entité Customer représente les acheteurs de la plateforme. Elle embarque une logique métier riche : le mode de paiement mobile par défaut (Mvola), une limite de crédit exprimée en Ariary (Ar), un encours de dette actuel, et un statut premium qui déclenche des règles commerciales spécifiques. Le numéro de téléphone est indexé et unique, car il sert de clé d'identification pour le paiement mobile.

4.1.3 Entité Product

L'entité Product gère le catalogue de produits. Elle distingue le prix de vente (price) et le prix d'achat (purchase_price), permettant le calcul de la marge. Le stock_quantity est décrémenté à chaque validation de commande, et safety_stock définit le seuil en dessous duquel une alerte est générée.

4.1.4 Entités Order et OrderItem

La relation entre Order et OrderItem modélise un panier d'achat. Une commande peut contenir plusieurs articles (relation one-to-many via SQLAlchemy relationship). Le montant total est calculé à la création en agrégeant les lignes. Le statut de la commande suit une machine d'états (PENDING → VALIDATED → DELIVERED) contrôlée par les endpoints.

4.1.5 Entités StockMovement et AuditLog

StockMovement enregistre chaque mouvement de stock (entrée ou sortie), horodaté et associé à l'utilisateur responsable. AuditLog est une table générique d'audit qui journalise toutes les actions métier significatives avec leur contexte, permettant une traçabilité complète du système.

4.2 Règles de Gestion Avancées

4.2.1 Règle de Limite de Crédit (RG-01)

Avant toute création de commande, le système vérifie que la dette actuelle du client ne dépasse pas sa limite de crédit. Si cette condition n'est pas satisfaite, la commande est bloquée et l'action est journalée avec le libellé BLOCKED_ORDER.

```
# Règle RG-01 : Contrôle limite de crédit
if customer.current_debt > customer.credit_limit:
    log_action(db, "BLOCKED_ORDER", f"Customer {customer.id}
exceeded credit limit.")
    raise HTTPException(400, "Order blocked: Credit limit
exceeded.")
```

4.2.2 Règle de Remise Client Premium (RG-02)

Les clients ayant le statut is_premium bénéficient automatiquement d'une remise de 10% sur tous les articles de leur commande. Cette remise est calculée à la création de l'OrderItem et persistée dans le champ discount_applied pour l'historique.

```
# Règle RG-02 : Remise client premium
discount = 0.10 if customer.is_premium else 0.0
final_price = product.price * (1 - discount)
order_item = OrderItem(..., discount_applied=discount)
```

4.2.3 Règle de Vérification de Stock (RG-03)

Lors de la validation d'une commande, le système vérifie que le stock disponible est suffisant pour chaque article. Si un article manque, la validation est refusée avec un message explicite et l'événement est journalé (STOCK_ERROR).

4.2.4 Règle d'Alerte Stock de Sécurité (RG-04)

Après chaque déduction de stock, le système compare la quantité restante au stock de sécurité défini par produit. Si le seuil est franchi, une alerte ALERT_STOCK est générée dans l'AuditLog, permettant une réapprovisionnement pro-actif.

```
# Règle RG-04 : Alerte stock de sécurité
product.stock_quantity -= item.quantity
if product.stock_quantity < product.safety_stock:
    log_action(db, "ALERT_STOCK", f"Product {product.sku}
below safety stock!")
```

4.2.5 Règle de Mise à Jour de la Dette Client (RG-05)

A la validation d'une commande, le montant total est ajouté à l'encours de dette du client (current_debt). Ce mécanisme permet un suivi en temps réel de l'exposition au risque par client, dont la consultation alimente la règle RG-01 pour les commandes suivantes.

4.3 Gestion des Transactions et de la Cohérence

L'ERP utilisé des transactions SQLAlchemy pour garantir l'atomicité des opérations. En cas d'échec à n'importe quelle étape du processus de création ou de validation d'une commande, un rollback automatique annule toutes les modifications partielles. Le pattern try/except/finally avec db.rollback() et db.close() assure la cohérence de la base de données.

4.4 Workflow de Commande

Le cycle de vie d'une commande dans le système suit les étapes suivantes :

- Étape 1 — Création (POST /orders/) : Validation client, calcul des prix, application des remises, réservation. Statut : PENDING
- Étape 2 — Validation (PUT /orders/{id}/validate) : Vérification stock, déduction physique, alerte sécurité, mise à jour dette. Statut : VALIDATED
- Étape 3 — Audit automatique : À chaque étape, une entrée dans AuditLog est créée avec l'action, le détail, l'utilisateur et l'horodatage

Cette architecture de workflow garantit la traçabilité complète de chaque commande, depuis sa création jusqu'à son exécution, répondant aux exigences d'auditabilité et de conformité.

5. Architecture Orientée Services (SOA)

5.1 Principes SOA Appliqués

Notre architecture s'inspire des principes fondamentaux du SOA : chaque service est un unité fonctionnelle autonome, exposant une interface standardisée et communicant exclusivement via des mécanismes bien définis. Les huit principes SOA de Thomas Erl ont guidé nos choix architecturaux :

- Contrat de service standardisé : Toutes les API sont documentées via Swagger (FastAPI /docs)
- Découplage : Chaque service a sa propre base de code, ses propres dépendances et son propre cycle de déploiement
- Abstraction : L'implémentation interne (ORM, modèles ML) est cachée derrière les endpoints REST
- Réutilisabilité : L'Analytics Service est consommé par le Dashboard et peut être intégré à tout système externe
- Composabilité : Le Dashboard orchestre les appels à l'ERP, l'ETL et l'Analytics Service
- Autonomie : Chaque service peut être redémarré ou mis à jour indépendamment
- Sans état (Stateless) : Les API REST ne maintiennent pas d'état entre les appels
- Discoverability : La documentation Swagger générée automatiquement rend les services découvrables

5.2 Détail des Services Exposés

5.2.1 Service ERP (Port 8000)

Le service ERP est le service noyau du système. Il expose les endpoints suivants :

Méthode	Endpoint	Description
GET/POST	/seed/	Initialisation des données maîtres
GET/POST	/seed_massive/	Génération de données volumiques pour l'IA
POST	/orders/	Création d'une commande (avec règles métier)
PUT	/orders/{id}/validate	Validation et déduction de stock

5.2.2 Service ETL (Port 8002)

Le service ETL gère le pipeline de données entre l'ERP et le Data Warehouse. Il expose :

Méthode	Endpoint	Description
GET	/	Health check du service ETL
GET	/status	Statut du dernier ETL (lignes chargées, heure)
GET/POST	/trigger	Déclenchement manuel de l'ETL

5.2.3 Service Analytics (Port 8001)

Le service Analytics est le moteur d'intelligence analytique du système. Consommé par le Dashboard Streamlit, il fournit :

Méthode	Endpoint	Description
GET	/mining/rfm	Segmentation RFM par K-Means (3 clusters)
GET	/mining/predictions	Prévisions ARIMA sur 90 jours
GET	/kpis	Indicateurs clés (CA total, marge) pour l'IA

5.3 Communication Inter-Services

Les communications entre services suivent un modèle synchrone basé sur des requêtes HTTP REST. Le Dashboard Streamlit agit comme orchestrateur en consommant simultanément le service Analytics et en présentant les résultats. La librairie (requests) de Python gère les appels HTTP depuis le Dashboard vers les API.

Dans l'environnement Docker, les services se référencent par leur nom de service réseau (analytics, etl, db), résolus par le DNS interne de Docker Compose. Les variables d'environnement ANALYTICS_API_URL et DATABASE_URL permettent la reconfiguration sans modification du code.

5.4 Documentation et Contrats d'Interface

FastAPI génère automatiquement une documentation Swagger UI accessible sur le chemin /docs de chaque service. Cette documentation constitue le contrat formel de chaque service, décrivant les paramètres d'entrée (DTO Pydantic), les codes de retour et les schémas de réponse. Les DTOs (Data Transfer Objects) implémentés avec Pydantic garantissent la validation automatique des données entrantes et la sérialisation JSON cohérente.

6. Business Intelligence, Data Mining et Reporting

6.1 Architecture du Data Warehouse

6.1.1 Choix du Modèle Dimensionnel

Nous avons choisi un schéma en étoile (Star Schema) comme modèle de données pour le Data Warehouse. Ce choix est justifié par sa simplicité de compréhension pour les utilisateurs métier, ses performances optimales pour les requêtes analytiques (peu de jointures), et sa compatibilité native avec les outils BI comme Metabase.

Le schéma en étoile implémenté comprend une table de faits centrale (fact_ventes) et quatre tables de dimensions gravitant autour :

Table	Type	Attributs clés
fact_ventes	Table de Faits	order_id, date_key, customer_id, product_id, magasin_id, quantity, montant_ht, marge
dim_temps	Dimension Temps	date_key, annee, mois, jour, saison
dim_client	Dimension Client	customer_id, customer_name, is_premium, segment, geographie
dim_produit	Dimension Produit	product_id, sku, product_name, purchase_price
dim_magasin	Dimension Magasin	magasin_id, nom_magasin, ville

6.1.2 Mesures de la Table de Faits

La table fact_ventes centralise trois mesures calculables pour l'analyse :

- montant_ht : Chiffre d'affaires hors taxes (quantité x prix unitaire après remise)
- cout_total : Coût d'achat total (quantité x prix d'achat), permettant le calcul de la rentabilité
- marge : Marge brute par ligne de vente (montant_ht - cout_total), indicateur clé de performance

6.1.3 Enrichissements Dimensionnels

La dimension temps a été enrichie d'un attribut saison calculaté automatiquement depuis le mois (Hiver, Printemps, Été, Automne), permettant des analyses saisonnières. La dimension client a été enrich ie d'un attribut segment (Premium/Standard) et d'une dimension géographique (Madagascar), anticipant l'extension future vers le multi-pays.

6.2 Pipeline ETL

6.2.1 Phase d'Extraction

L'extraction récupère les commandes validées (status = 'VALIDATED') depuis l'ERP via une jointure SQL multi-tables (orders, order_items, products, customers). Cette condition filtrante garantit que seules les données finalisées alimentent le Data Warehouse, préservant la qualité analytique.

```
SELECT o.id, o.created_at, oi.quantity, oi.unit_price,
       p.sku, p.purchase_price, c.name, c.is_premium
FROM orders o
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
JOIN customers c ON o.customer_id = c.id
WHERE o.status = 'VALIDATED'
```

6.2.2 Phase de Transformation

La phase de transformation est implémentée avec Pandas et comprend plusieurs étapes :

- Construction des tables de dimensions par extraction et déduplication des attributs
- Calcul des mesures dérivées (montant_ht, cout_total, marge)
- Enrichissement de la dimension temps avec l'année, le mois, le jour et la saison
- Enrichissement de la dimension client avec le segment et la géographie
- Création de la dimension magasin avec une valeur par défaut (Online Marketplace)

6.2.3 Phase de Chargement

Le chargement utilise la méthode Pandas to_sql() avec l'option if_exists='replace', implémentant un chargement complet (full load). Cette stratégie garantit la cohérence des données pour les petits volumes. Le scheduler Python exécute le pipeline toutes les minutes, avec un endpoint /trigger pour les déclenchements à la demande.

6.3 Data Mining

6.3.1 Segmentation RFM par K-Means

La segmentation RFM (Récence, Fréquence, Montant) est une technique éprouvée du marketing analytique qui permet de classer les clients en groupes homogènes selon leur comportement d'achat. Notre implémentation en trois étapes :

Calcul des indicateurs RFM : La récence est calculée en jours depuis le dernier achat (inversement proportionnelle à la fidélité), la fréquence est le nombre de commandes distinctes, et le montant total est le chiffre d'affaires cumulé par client.

Normalisation et clustering : Les trois dimensions RFM sont normalisées via StandardScaler (centrage-réduction) pour éviter que des échelles différentes ne biaisen les résultats. L'algorithme K-Means avec k=3 est ensuite appliqué avec 10 initialisations aléatoires (n_init=10) pour trouver le meilleur optimum local.

Labellisation automatique : Les trois clusters sont automatiquement étiquetés selon le montant moyen dépensé : le cluster avec le montant le plus faible est 'At Risk', le médian est 'Occasional', et le plus fort est 'Premium'. Cette labellisation automatique garantit la cohérence indépendamment de l'ordre des clusters retournés par K-Means.

> *Valeur métier : La segmentation RFM permet de cibler les actions commerciales — réengagement des clients 'At Risk', programme de fidélité pour les 'Occasional', offres exclusives pour les 'Premium'.*

6.3.2 Prévision des Ventes par ARIMA

La modélisation ARIMA (AutoRegressive Integrated Moving Average) est la technique de référence pour l'analyse et la prévision des séries temporelles. Notre implémentation prévoit les ventes sur un horizon de 90 jours (3 mois).

Le modèle ARIMA(1,1,1) a été sélectionné avec les paramètres suivants : p=1 (autoregressive order) pour capturer la dépendance au jour précédent, d=1 (differencing order) pour rendre la série stationnaire, q=1 (moving average order) pour lisser les fluctuations aléatoires.

Un mécanisme de fallback a été implémenté : si l'historique contient moins de 5 observations, le système retourne des prévisions de simulation (mock data) pour ne pas bloquer la démonstration. Les prévisions journalières sont ensuite agrégées au niveau mensuel pour une lecture décisionnelle plus adaptée.

6.4 Tableaux de Bord

6.4.1 Dashboard Streamlit

Le dashboard Streamlit est organisé en trois onglets correspondant aux trois dimensions analytiques du système :

- Onglet Segmentation Client : Tableau de données RFM et nuage de points interactif (récence vs fréquence, taille proportionnelle au montant, couleur par segment)
- Onglet Prédictions : Graphique linéaire des ventes prévues sur 3 mois avec courbe lissée
- Onglet Reporting IA : Interface de génération de rapport narratif automatisé

6.4.2 Metabase (BI Self-Service)

Metabase est intégré comme outil de BI self-service, connecté directement au Data Warehouse bi_warehouse. Il permet aux utilisateurs non-techniciens d'explorer les données sans écrire de SQL, de créer des tableaux de bord personnalisés, et de configurer des alertes automatiques. La base metabase_app_db stocke les configurations, tableaux de bord et réglages de Metabase.

7. Reporting Assisté par l'Intelligence Artificielle

7.1 Architecture du Module IA

Le module de reporting assisté par IA constitue le point culminant de la chaîne décisionnelle. Il combine la collecte automatisée des indicateurs clés de performance (KPIs) avec la génération de langage naturel (NLG — Natural Language Generation) via un grand modèle de langage.

Notre implémentation repose sur l'API Groq, qui fournit un accès ultra-rapide au modèle Llama 3.1 8B (llama-3.1-8b-instant). Le choix de Groq est justifié par sa rapidité d'inférence exceptionnelle (tokens/s parmi les plus élevés du marché), son tier gratuit généreux pour le développement, et la qualité de réponse du modèle Llama 3.

7.2 Flux de Génération du Rapport

La génération d'un rapport suit un pipeline en quatre étapes :

- Étape 1 — Collecte des KPIs : Appel à l'endpoint GET /kpis du service Analytics, retournant le CA total et la marge totale. Une valeur de référence (ca_annee_derniere) est utilisée pour calculer le taux de croissance.
- Étape 2 — Calcul des indicateurs dérivés : Calcul du taux de croissance en pourcentage ((CA_actuel - CA_precedent) / CA_precedent * 100), transformé en variable quantitative prête à l'injection dans le prompt.
- Étape 3 — Génération du rapport : Le prompt structuré injecte les valeurs calculées et demande explicitement un rapport sans variables non résolues, garantissant un texte directement exploitable.
- Étape 4 — Affichage et fallback : Le rapport généré est affiché dans l'interface. En cas d'échec API, un message de fallback affiche les valeurs brutes.

7.3 Ingénierie du Prompt

La qualité du rapport généré repose sur un prompt bien structuré. Voici l'approche adoptée :

```
Rédige un rapport financier court avec ces données :
- CA Actuel : {ca}€
- Marge : {marge}€
- Croissance par rapport à l'an dernier : {croissance:.1f}%
Ne laisse pas de variables comme 'X%' dans le texte.
```

Cette instruction explicite ('Ne laisse pas de variables comme X%') est une technique d'ingénierie de prompt (négative prompting) qui force le modèle à produire un texte entièrement résolu et directement lisible par un manager.

7.4 Capacités de Génération

Le module IA est capable de générer automatiquement plusieurs types de contenu décisionnel :

7.4.1 Insights Automatiques

Le LLM interprète les chiffres dans leur contexte et génère des observations pertinentes : identification des tendances positives ou négatives, comparaison avec les périodes précédentes, mise en perspective des écarts.

7.4.2 Narration des Résultats (Data Storytelling)

Plutôt que de présenter des chiffres bruts, le module transforme les indicateurs en narration fluide et compréhensible par tout décideur, même non spécialisé en finance ou en data science.

7.4.3 Recommandations Décisionnelles

Sur la base des tendances détectées, le LLM peut formuler des recommandations d'action : ajustement des stocks, offres commerciales ciblées, révision des objectifs de vente.

7.5 Limites et Améliorations Possibles

L'implémentation actuelle présente certaines limites. La dépendance à l'API externe Groq introduit un risque de disponibilité. Le prompt n'injecte que deux métriques (CA et marge), alors qu'une version avancée devrait intégrer les résultats RFM, les prévisions ARIMA et les alertes de stock. Enfin, le taux de croissance est calculé par rapport à une valeur hardcodeée (40 millions Ar), qui devrait être récupérée dynamiquement depuis le Data Warehouse.

8. Analyse Critique et Limites

8.1 Points Forts du Système

8.1.1 Architecture Technique

L'architecture Docker-based garantit une reproductibilité parfaite de l'environnement, éliminant les problèmes de dépendances. La séparation stricte entre base opérationnelle (erp_db) et analytique (bi_warehouse) est une bonne pratique fondamentale qui évite toute interférence entre les charges de travail OLTP et OLAP. Le choix de FastAPI est excellent : haute performance (basé sur Starlette/ASGI), validation automatique via Pydantic, et documentation Swagger auto-générée.

8.1.2 Règles de Gestion

Les règles métier implémentées (contrôle crédit, remise premium, alerte stock) sont directement applicables dans un contexte réel. Le système d'audit log est complet et permet une traçabilité fine des actions. La gestion transactionnelle avec rollback automatique garantit la cohérence des données même en cas d'échec partiel.

8.1.3 Chaîne Décisionnelle

La chaîne ETL → Data Warehouse → Analytics → Dashboard IA est complète et fonctionnelle. La combinaison RFM + ARIMA offre une vision duale : descriptive (qui sont mes clients ?) et prédictive (quelles seront mes ventes ?). L'intégration du LLM pour le reporting narratif constitue un apport différenciateur significatif.

8.2 Limites Identifiées

8.2.1 Limites Techniques

Chargement ETL full-load : La stratégie if_exists='replace' recharge intégralement les tables à chaque ETL. Cela est acceptable pour les volumes actuels, mais deviendra un goulot d'étranglement à grande échelle. Une stratégie de chargement incrémental (basée sur un watermark timestamp) serait nécessaire en production.

Absence de gestion des erreurs distribuées : Si le service Analytics est indisponible, le Dashboard affiche simplement une erreur. Un pattern circuit breaker ou une file de messages (type Kafka/RabbitMQ) améliorerait la résilience.

Sécurité des API : Les endpoints ERP ne nécessitent pas d'authentification (pas de JWT ni d'OAuth). En production, chaque appel API devrait être authentifié et autorisé.

8.2.2 Limites du Module IA

Dépendance externe : L'intégration Grok introduit une dépendance à un service tiers. Une défaillance de l'API Grok rend le module de reporting IA inopérant. Le mécanisme de fallback (affichage des valeurs brutes) atténue partiellement ce risque.

Contexte limité du LLM : Le prompt actuel n'injecte que CA et marge. Les informations sur la segmentation client, les anomalies de stock ou les tendances saisonnières ne sont pas exploitées. Un prompt enrichi produirait des rapports de bien meilleure qualité décisionnelle.

Valeur de référence hardcode : Le CA de l'année précédente (ca_annee_derniere = 40 000 000 Ar) est une constante dans le code, ce qui fausse le calcul du taux de croissance pour les jeux de données réels.

8.2.3 Limites du Data Mining

Choix fixe de k=3 clusters : Le nombre de clusters K-Means est fixé à 3 sans analyse préalable (méthode du coude, silhouette score). Pour un jeu de données plus important, le k optimal pourrait être différent. Une sélection automatique du k optimal améliorerait la robustesse.

Paramétrage ARIMA fixe : Les paramètres (1,1,1) ont été choisis manuellement. Un processus d'auto-arima (librairie pmdarima) sélectionne automatiquement les meilleurs paramètres p, d, q par minimisation de l'AIC.

8.3 Comparaison avec les Exigences du Projet

Exigence	Statut	Commentaire
ERP et règles de gestion	✅ Complet	5 règles métier implémentées
Architecture SOA	✅ Complet	3 microservices FastAPI REST
Data Warehouse	✅ Complet	Star Schema 4 dimensions
ETL automatisé	✅ Complet	Scheduler 1 minute + trigger manuel
Data Mining	✅ Complet	RFM K-Means + ARIMA
Reporting IA (NLG)	✅ Complet	LLM Llama 3.1 via Groq
Tableaux de bord	✅ Complet	Streamlit + Metabase
Sécurité des API	⚠️ Partiel	Pas d'authentification JWT
ETL incrémental	⚠️ Partiel	Full load uniquement

9. Conclusion et Perspectives

9.1 Bilan du Projet

Ce projet a permis de concevoir et d'implémenter un système d'information intégré complet, allant de la gestion opérationnelle des transactions e-commerce jusqu'à la génération automatique de rapports stratégiques par Intelligence Artificielle. La plateforme Madagascar E-Commerce incarne les enjeux réels du secteur commercial malgache : paiements mobiles, gestion de crédit, ventes multi-produits et besoin croissant de pilotage data-driven.

L'ensemble des exigences fonctionnelles du projet d'examen ont été couvertes : un ERP transactionnel avec cinq règles de gestion avancées, une architecture SOA avec trois microservices découplés, un Data Warehouse en schéma en étoile alimenté par un ETL automatisé, des techniques de Data Mining (K-Means RFM et ARIMA), des tableaux de bord interactifs (Streamlit et Metabase), et un module de reporting narratif assisté par le LLM Llama 3.1.

9.2 Apports Pédagogiques

Ce projet a mobilisé un large éventail de compétences techniques et méthodologiques. Sur le plan technique, nous avons maîtrisé FastAPI pour la construction d'API REST robustes, SQLAlchemy ORM pour la persistance relationnelle, Pandas pour le traitement de données en batch, Scikit-learn pour le Machine Learning, et l'intégration d'API LLM externes via Grok.

Sur le plan méthodologique, ce projet nous a enseigné l'importance de la séparation des responsabilités dans la conception logicielle, la valeur de la documentation automatisée, et la complémentarité entre les approches descriptive (RFM) et prédictive (ARIMA) dans l'analyse des données commerciales.

9.3 Perspectives d'Évolution

9.3.1 Évolutions Techniques Court Terme

- Ajout d'une couche d'authentification JWT sur tous les endpoints ERP
- Migration vers un ETL incrémental avec gestion de watermark
- Implémentation d'auto-ARIMA pour la sélection automatique des paramètres
- Amélioration du prompt IA pour intégrer les résultats RFM et les alertes de stock

9.3.2 Évolutions Fonctionnelles Moyen Terme

- Extension du module ERP aux retours produits et au processus SAV
- Intégration native des API Mvola, Orange Money et Airtel Money pour la vérification des paiements en temps réel
- Mise en place d'un module de recommandation produit par filtrage collaboratif
- Développement d'un chatbot analytique interactif pour les managers

9.3.3 Scalabilité et Production

- Migration vers une architecture Kubernetes pour l'orchestration en production
- Séparation physique des instances PostgreSQL ERP et BI Warehouse
- Implémentation d'un bus de messages (Apache Kafka) pour les communications asynchrones entre services
- Mise en place d'une stratégie de backup automatisé et de disaster recovery

9.4 Conclusion

Ce projet démontre qu'il est possible de construire un système d'information d'entreprise complet, moderne et intelligent avec des technologies open source exclusivement. La stack retenue (FastAPI, PostgreSQL, Pandas, Scikit-learn, Streamlit, Metabase, Groq/Llama) offre un excellent équilibre entre simplicité de développement, robustesse et capacités analytiques avancées.

La dimension malgache du projet — avec ses spécificités de paiement mobile, sa monnaie nationale (Ariary) et son contexte de commerce local — apporte une ancrage concret qui va bien au-delà d'un exercice académique. Ce type de plateforme a une valeur réelle pour les PME de distribution malgaches en pleine numérisation.

> *"La donnée est le nouveau pétrole, mais contrairement au pétrole, elle s'enrichit à mesure qu'on la partage." — Ce projet illustre comment transformer les données opérationnelles brutes en intelligence décisionnelle au service des entreprises malgaches.*